Exercises · Generative AI and LLM Fundamentals

AI02 Exercises

Work each one before reading its solution. Practise Exercise 6 out loud. Prices are from the October 2026 reference used in Lesson 7, so check them before relying on them.

Exercise 1: Train BPE by hand

The corpus is `low low low lower lower newest newest newest`. Treat each word as characters with `_` marking its end.

(a) Count the word frequencies. **(b)** Name the most frequent adjacent pair, and merge it. **(c)** What does this show about why tokenizers do well on common text?

Solution

(a) low 3, lower 2, newest 3.

(b) Count pairs: `l o` appears 5 times (low 3 + lower 2), `o w` appears 5 times, and `w e` appears 5 times (lower 2 + newest 3). So there's a three-way tie at 5. Implementations break ties by a fixed rule, so any of these is a valid answer. Merging `l o` gives the symbol `lo`.

(c) Frequent sequences merge first, so common text reaches single tokens quickly. Text unlike the corpus stays fragmented, costing more tokens.

Exercise 2: A token budget

Your prompt measures 800,000 tokens on an older model. You're moving to a Claude 4.7+ model with a 1M window, and you reserve 16,000 tokens for output.

(a) Using Anthropic's approximate 30% figure, estimate the new count. **(b)** Does it fit? **(c)** What should you actually do?

Solution

(a) 800,000 × 1.3 = **1,040,000**.

(b) No. It exceeds 1M before you reserve output. 1,040,000 + 16,000 = 1,056,000.

(c) Don't rely on the estimate. Call `count_tokens` with the target model, since the increase depends on content. Then trim, send relevant parts only, or split the work. A prompt this large is probably also a design problem (AI03 L6).

Exercise 3: Attention costs

(a) How many more attention pairs does a 20,000-token input have than a 2,000-token input? **(b)** Does this mean the API bill is 100 times higher? **(c)** So why does the number still matter?

Solution

(a) (20,000 / 2,000)² = **100 times** as many.

(b) No. API prices are generally per token, so input charges rise about 10 times, not 100.

(c) It explains the physical pressure on providers, and it's why long context can increase latency and memory use. The KV cache also grows with context (HLD M21D). It's a reason to keep prompts focused, even where prices are linear.

Exercise 4: Streaming or not

TTFT is 1.0 s, and output speed is 80 tokens per second. **(a)** Calculate the total time for 1,200 and 300 output tokens. **(b)** A product manager says, "Streaming won't help because total time doesn't change." Respond.

Solution

(a) 1,200 tokens: 1.0 + 15 = **16 s**. 300 tokens: 1.0 + 3.75 = **4.75 s**.

(b) That's correct for total time, and it misses what users perceive. With streaming, they see the first words after 1 second, rather than a blank screen for 16 seconds. For a chat UI, that's the metric that matters. For a backend pipeline nobody watches, the PM has a point, since streaming mainly adds complexity there, apart from guarding against timeouts on very long outputs.

Exercise 5: Batch or synchronous

A nightly job sends 2 million requests, each with 5,000 input and 800 output tokens, using Opus 5.5.

(a) Compute the full-price cost. **(b)** Compute the batch cost. **(c)** Which workloads in your system shouldn't move to batch?

Solution

(a) Per request: (5,000 × $4 + 800 × $20) / 1M = $0.036. × 2M = **$72,000**.

(b) 50% less = **$36,000**.

(c) Anything blocking a person or a process, such as a chat reply or a pre-merge check, since batch has no latency guarantee and can take up to 24 hours. Also anything needing multi-turn tool calls inside one request, which batch doesn't support.

Exercise 6: Defend a decision

Your CTO says: "Claude is trained with Constitutional AI, and it has a safety system. So we don't need approval steps on the refund tool, and that saves us months of work." Write the answer you'd give out loud.

Model answer

"You're right that Claude's training gives us strong defaults, and I wouldn't rebuild that. It's a real benefit, and we can lean on it for general behavior.

But training shapes probabilities. It can't know our refund limits or who's authorized, and Anthropic's own research shows that behavior can be steered, and that a model's stated reasoning may not match what happened internally. So it can't be our control for money movement.

My rule is that model training and prompts handle judgment, and code handles guarantees. For refunds, that means a hook that blocks anything above the limit, and human approval above a threshold. It's far less than months of work, because it's a validation layer, not a platform.

I'd change my mind on the approval step if we measured very low-value refunds, where the cost of review exceeds the risk. Then I'd keep the hard limit in code, remove the human step below a threshold, and keep sampling those decisions."

Why this works: it accepts the valid premise, gives the mechanism, offers a concrete and smaller alternative, and states what would change your mind.

Your notes